

# Python Topic 9 — Loops (for / while)

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert



## Layer 1 — New to Python

A loop repeats code. Python's `for` loop goes **through items directly**, not counting with an index.

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)           # apple, banana, cherry
```

Read it like English: "for each **fruit** in fruits, print it." No `i = 0; i < length; i++` counting.

To loop a certain number of times, use `range()` :

```
for i in range(5):
    print(i)               # 0, 1, 2, 3, 4 (starts at 0, stops before 5)
```

A `while` loop repeats as long as a condition is true:

```
count = 0
while count < 3:
    print(count)           # 0, 1, 2
    count += 1             # must change count, or it loops forever!
```



## Layer 2 — Intermediate (real code + how it works)

The `for` loop is a "for-each"

```
// JS - C-style counting
for (let i = 0; i < arr.length; i++) {
    console.log(arr[i]);
}

# Python - iterate items directly
for item in arr:
    print(item)
```

## `range()` — three forms

```
range(5)           # 0,1,2,3,4           → stop only
range(2, 8)        # 2,3,4,5,6,7        → start, stop
range(0, 10, 2)    # 0,2,4,6,8          → start, stop, step
range(10, 0, -1)   # 10,9,8...1         → countdown (negative step)
```

**i** **Lazy:** `range` doesn't build a list in memory — it generates numbers on demand (efficient for big ranges).

## `enumerate` — when you need the index too

```
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")      # 0: apple, 1: banana, ...

for i, fruit in enumerate(fruits, start=1):
    print(f"{i}: {fruit}")      # 1: apple, 2: banana, ... (custom start)
```

Replaces JS's `arr.forEach((item, i) => ...)`.

## `zip` — loop over two lists together

```
names = ["Akshay", "Riya"]
ages = [26, 24]

for name, age in zip(names, ages):
    print(f"{name} is {age}")    # Akshay is 26, Riya is 24
```

No index juggling. `zip` stops at the shorter list.

## `break` and `continue` (same as JS)

```
for n in range(10):
    if n == 5:
        break          # exit the loop entirely
    if n % 2 == 0:
        continue       # skip to next iteration
    print(n)           # 1, 3
```

## Looping a dict (recall Topic 7)

```
person = {"name": "Akshay", "age": 26, "city": "Pune"}

for key, value in person.items():
```

```
print(f"{key}: {value}")  
# name: Akshay  
# age: 26  
# city: Pune
```

## Layer 3 — Expert (internals & gotchas)

### 1. `for...else` — Python's surprising feature (no JS equivalent)

```
for n in numbers:  
    if n == target:  
        print("Found!")  
        break  
else:  
    print("Not found")    # runs ONLY if the loop finished WITHOUT break
```

The `else` on a loop runs when the loop completes normally (no `break`). Perfect for search patterns — "did we find it or not?" Same works with `while...else`.

### 2. Don't modify a list while looping over it

```
# ❌ BUG – modifying during iteration skips items  
nums = [1, 2, 3, 4]  
for n in nums:  
    if n % 2 == 0:  
        nums.remove(n)    # corrupts the iteration!  
  
# ✅ Loop over a copy, or use a comprehension  
nums = [n for n in nums if n % 2 != 0]    # the Pythonic way
```

### 3. Prefer comprehensions over building lists in loops

```
# ❌ verbose loop  
result = []  
for n in range(10):  
    result.append(n * 2)  
  
# ✅ comprehension – faster, idiomatic  
result = [n * 2 for n in range(10)]
```

✅ **Rule:** use a loop for side effects (printing, API calls); use a comprehension when building a collection.

#### 4. `while True` + `break` — the idiomatic infinite loop

```
while True:
    command = input("Enter command: ")
    if command == "quit":
        break
    process(command)
```

Python has no `do...while` ; this pattern replaces it.

#### 5. Looping gotcha — late binding in closures

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)      # all capture the SAME i
print([f() for f in funcs])     # [2, 2, 2] ← not [0, 1, 2]!

# Fix: bind i as a default argument
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs])     # [0, 1, 2] ✓
```

⚠ **Note:** the closure captures the *variable*, not its value at that moment. Same category of bug as old JS with `var` .

#### 6. Iterating efficiently — generators for big data

```
# For huge datasets, don't build a list – use a generator (lazy)
def read_lines(file):
    for line in file:          # yields one line at a time, low memory
        yield line.strip()
```

We'll cover generators deeply later. `for` works on any *iterable* — lists, dicts, files, generators, ranges — all with the same syntax.

### Interview Q&A Cards

**Q:** How is Python's `for` different from JS's?

**A:** It's a for-each over any iterable, not index-counting. Use `range` for counts, `enumerate` for index+value, `zip` for parallel lists.

**Q:** What does `for...else` do?

A: The `else` block runs only if the loop finished without hitting `break`. Ideal for "searched everything and didn't find it" logic.

#### Q: Why not modify a list while iterating it?

A: Removing/adding shifts indices under the iterator, skipping or repeating items. Iterate over a copy, or build a new list with a comprehension.

#### Q: Why do loop-created lambdas all return the same value?

A: Late binding — the closure captures the loop variable, read at call time (its final value). Fix by binding per-iteration, e.g. `lambda i=i: i`.

🔗 **Memory hook:** `for x in thing` — no counters. `enumerate` gives index, `zip` pairs lists, `for...else` means "loop finished without break".

✅ **One-liner for interviews:** "Python's `for` is a for-each over any iterable — use `range` for counts, `enumerate` for index+value, `zip` for parallel lists. `for...else` runs the else when no `break` fired. Never mutate a list mid-iteration; prefer comprehensions for building collections."